# Automatic smart traffic control system using deep learning

**Name of Student: Sree**

**Department:**

**Registration No.:**

**Name of Research Supervisor:**

# Abstract

In this dissertation, I design an automatic smart traffic control system based on deep learning to solve congestion and inefficiencies in urban transportation networks. Two datasets were used, the City of Chicago traffic dataset with 19.5 million timestamped records, and a synthetic dataset that was created to reflect rare and extreme conditions. Data was preprocessed through cleaning, feature engineering and encoding to make the data compatible with sequential and feature-based models. The features were learned by temporal dependencies using Long Short-term Memory (LSTM) networks, and the predictions were made using XGBoost. The experimental analysis showed that LSTM was applicable to predicting gradual increases in traffic flow over time, but XGBoost was applicable to predicting sudden increases in congestion and unusual event occurrences. Generalization, strength, and predictive accuracy were enhanced in the two models by the synthesizing data. Low MAE and RMSE values as the result of statistical and error analysis proved reliability. The applicability of the models in peak, off-peak, and emergency situations was demonstrated in scenario-based evaluations. The results are unified to provide an urban mobility optimisation and real-time traffic control on a scalable, smart grid.

# Acknowledgement

I would like to say that I am very grateful to my supervisor, who has helped me throughout this research and supported me with the invaluable advice. I also owe it to my university to offer me the academic support that I needed. There is no way this project would have become possible without Google Colab to practice the project and Kaggle datasets to experiment with it. Finally, my colleagues and relatives have always encouraged me and helped me, that is why I would like to thank them.

# Table of Contents

Abstract ..... 2

Acknowledgement ..... 3

Chapter 1: Introduction ..... 12

1.0 Research Background ..... 12

1.1 Problem Statement ..... 13

1.2 Research Questions ..... 14

1.3 Aim and Objectives ..... 14

1.4 Expected Practical Element Output ..... 15

1.4.1 Methodology ..... 15

1.4.2 Project Outcome ..... 15

1.5 Project Plan – Gantt Chart ..... 16

Chapter 2: Literature Review ..... 17

2.1 Comprehensive Overview of the Existing Literature ..... 17

2.1.1 Evolution of Traffic Control Systems ..... 17

2.1.2 The Role of Deep Learning in Traffic Monitoring ..... 18

2.1.3 Object Detection and Vehicle Classification ..... 19

2.1.4 Adaptive Signal Control Using AI ..... 21

2.1.5 Real-Time Systems and Edge Computing ..... 23

2.1.6 Smart City Integration and IoT Synergy ..... 23

2.2 Critical Analysis of Existing Studies ..... 25

2.2.1 Limitations of Static and Sensor-Based Systems ..... 25

2.2.2 Challenges in Computer Vision-Based Approaches ..... 26

2.2.3 Trade-Offs in Deep Learning and Reinforcement Learning ..... 27

2.2.4 Integration and Interoperability Concerns ..... 28

2.2.5 Gaps in Research and Opportunities for Advancement ..... 28
2.3 Technological Foundations and Frameworks for Smart Traffic Control ..... 29
2.3.1 System Architecture of AI-Based Traffic Control ..... 29
2.3.2 Real-Time Processing Using Edge Devices ..... 31
2.3.3 Ethical and Privacy Considerations ..... 31
2.4 Summary ..... 32
Chapter 3: Methodology ..... 34
3.Research Approach ..... 34
3.1 Data Collection and Preprocessing ..... 34
3.1.1 Dataset Selection ..... 35
3.1.2 Data Cleaning and Integration ..... 35
3.1.3 Feature Engineering ..... 36
3.1.4 Data Preparation for Deep Learning ..... 36
3.2 Deep Learning Model Development ..... 36
3.2.1 Model of predicting traffic flow ..... 37
3.2.2 Adaptive traffic signal control algorithm ..... 37
3.2.3 Integration of models and system architecture ..... 37
3.3 Evaluation of the Proposed System ..... 38
3.3.1 Traffic Prediction Measures ..... 38
3.3.2 Traffic signal control metrics ..... 39
3.3.3 Comparative Testing ..... 39
3.4 Additional Methodological Considerations ..... 39
3.4.1 Practical Constraints ..... 39
3.4.2 Moral and Environmental Effect ..... 40
3.4.3 Required Resources ..... 41

3.5 Summary ..... 41
Chapter 4: Implementation and Development ..... 43
4.1 Data Acquisition and Preparation ..... 43
4.1.1 Synthetic Dataset Cleaning and Encoding ..... 44
4.1.2 Chicago Traffic Dataset Cleaning ..... 45
4.1.3 Feature Engineering and Encoding ..... 46
4.2 Sequence Creation for LSTM Models ..... 47
4.2.1 Sequence Construction Methodology ..... 47
4.2.2 Handling Missing Values in Sequences ..... 48
4.3 LSTM Model Architecture ..... 48
4.3.1 Training Procedure ..... 49
4.3.2 LSTM Performance Evaluation ..... 49
4.4 XGBoost Model for Comparison ..... 50
4.4.1 Feature Importance Analysis ..... 51
4.5 Model Integration and Simulation Considerations ..... 51
4.5.1 Traffic Scenario Simulation ..... 52
4.5.2 Integration Testing ..... 52
4.5.3 Performance Validation and Analysis ..... 53
4.6 Challenges and Mitigation Strategies ..... 53
4.6.1 Large Dataset Management ..... 53
4.6.2 Sequence Integrity and Alignment ..... 54
4.6.3 Feature Dimensionality and Overfitting ..... 54
4.7 Conclusion ..... 55
Chapter 5: Results and Evaluation ..... 56
5.1 Dataset Overview and Preprocessing ..... 56

5.1.1 Chicago Real-World Data ..... 56
5.1.2 Synthetic Traffic Dataset ..... 57
5.2 Predictive Performance Analysis ..... 58
5.2.1 LSTM Model Results ..... 59
5.2.2 XGBoost Model Results ..... 59
5.2.3 Comparative Analysis ..... 60
5.3 Scenario-Based Performance Evaluation ..... 61
5.3.1 Peak and Off-Peak Traffic ..... 62
5.3.2 Emergency Event Modeling ..... 62
5.4 Statistical and Error Analysis ..... 62
5.4.1 Feature Contribution Analysis ..... 63
5.4.2 Residual and Error Distribution ..... 64
5.5 Discussion on Synthetic vs. Real Data ..... 64
5.5.1 Contributions to Model Accuracy ..... 65
5.5.2 Limitations of Dataset Integration ..... 65
5.6 Visualization of Results ..... 66
5.6.1 Actual vs Predicted Traffic Counts Analysis ..... 66
5.6.2 Residuals Analysis ..... 67
5.6.3 Accuracy and Loss Plots ..... 68
5.7 Conclusion ..... 69
Chapter 6: Conclusion and Recommendations ..... 71
6.1 Conclusion ..... 71
6.2 Recommendations ..... 71
6.3 Future Work ..... 72
6.4 Key Findings and Contributions ..... 73

6.5 Concluding Remarks ..... 73

References ..... 75

Appendix ..... 83

**List of Figures**

Figure 1 Gantt Chart ..... 16
Figure 2 Cloud-based Vehicular Environment (Traffic Congestion Prediction). Source: (Abdullah et al., 2023) ..... 18
Figure 3 Image showing object detection operator mechanism Source: (Alsuwaylimi et al., 2024) ..... 19
Figure 4 The overall process of Traffic Congestion Prediction. Source: (Abdullah et al., 2023) 21
Figure 5 Data collection sources for smart transportation strategies. Source: (Ang et al., 2022) 22
Figure 6 Generic architecture for big data analytics in smart transportation. Source: (Ang et al., 2022) ..... 24
Figure 7 The flowchart of the proposed framework design for vehicle tracking based on a video surveillance system. Source: (Ait Abdelali et al., 2021) ..... 27
Figure 8 Architecture for AI based traffic control system Source: (Balasubramanian et al., 2023) ..... 30
Figure 9 System Architecture ..... 40
Figure 10 Dataset Shape and Implementation ..... 44
Figure 11 Synthetic Dataset Implementation ..... 44
Figure 12 Synthetic Dataset Cleaning and Encoding ..... 45
Figure 13 Chicago Traffic Dataset Cleaning ..... 46
Figure 14 Feature Engineering and Encoding Implementation ..... 47
Figure 15 LSTM Model Development ..... 49
Figure 16 LSTM Model Performance ..... 50
Figure 17 XGBoost Model Implementation ..... 51
Figure 18 Chicago Real-World Data Overview ..... 57
Figure 19 Synthetic dataset overview ..... 58
Figure 20 LSTM Model Performance ..... 58
Figure 21 XGBoost Performance ..... 59
Figure 22 LSTM Model Results ..... 59
Figure 23 XGBoost Model Results ..... 60
Figure 24 Comparison Model Result ..... 61
Figure 25 Statistic representation ..... 64

Figure 26 Actual vs Predicted Traffic Counts Result ..... 67
Figure 27 Residuals Result ..... 68
Figure 28 Model Accuracy plot ..... 69
Figure 29 Model Loss Result ..... 69

**List of Table**

Table 1 Smart Traffic Control Using Deep Learning.....**Error! Bookmark not defined.**

# Chapter 1: Introduction

## 1.0 Research Background

The proposed project is developing an intelligent traffic control system enabled with Artificial Intelligence to monitor the density of vehicles and optimize traffic lights cycle in real-time with deep learning and computer vision. The system will utilize live data using street cameras and a labeled Kaggle-dataset of traffic situations and automatically regulate traffic lights in real-time to even out traffic in the street. The idea is to decrease waiting time, fuel usage and enhance transit of emergency vehicles without the need to increase and decrease it manually (Saleem et al., 2022). The last product is a dynamic traffic control system that can be deployed, which can respond to changes in traffic in cities.

Deep learning, which is a subset of Artificial Intelligence, has changed intelligent transportation systems by offering automation beyond the rule-based traffic administration. CNNs and, in general, deep neural networks have proven to be very effective in the object detection and classification of images (Sharma et al., 2021). The abilities rendered them apt to real-time monitoring and control of traffic. The traffic system powered by AI can identify the vehicle numbers, distinguish them, whether it is a bike, car, or bus and can dynamically make decisions to control (traffic light) thus ensuring minimal input of human supervision.

Conventionally, however, the control of the traffic lights is carried out by relatively obsolete timers or simple sensors that are not adaptive to the current traffic. Such systems tend to lead to unnecessary delays and congestions especially at off-peak times or in instances of exceptional traffic occurrences (Sheikh and Peng, 2022). Additionally, they are also not flexible and are ineffective in dealing with traffic anomalies such as accidents and road blocks or emergency cases. Most notably, a deep learning system will learn patterns in past and comparative real-time data and, therefore, is smart enough to adapt itself to this change.

The essence of this study is in the use of a real-time traffic classification scheme that is applied using deep learning. Having the system trained on an extensive set of traffic video (and/or image) data provided by Kaggle, the model will be able to recognize the level of traffic density and determine the ideal length of green, yellow, and red traffic lights. The model will be implemented

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

| Task | Week 1 | Week 2 | Week 3 | Week 4 | Week 5 | Week 6 | Week 7 | Week 8 | Week 9 | Week 10 | Week 11 | Week 12 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Supervisor Allocation | | | | | | | | | | | | |
| Library session | | | | | | | | | | | | |
| **Milestone 1:- Topic selection** | | | | | | | | | | | | |
| Proposal preparation and review | | | | | | | | | | | | |
| **Milestone 2:- Proposal Submission** | | | | | | | | | | | | |
| Abstarct | | | | | | | | | | | | |
| **Chapter 1- Introduction** | | | | | | | | | | | | |
| **Chapter 2- Literature Review** | | | | | | | | | | | | |
| **Milestone 3:- Practical implementation/ Model Development** | | | | | | | | | | | | |
| **Chapter 3- Methodology** | | | | | | | | | | | | |
| **Chapter 4- Experimental Results Documentation** | | | | | | | | | | | | |
| **Chapter 5- Conclusion and Future work** | | | | | | | | | | | | |
| **Final review and Modifications** | | | | | | | | | | | | |
| **Milestone 4:- Final Submission** | | | | | | | | | | | | |

[redacted]

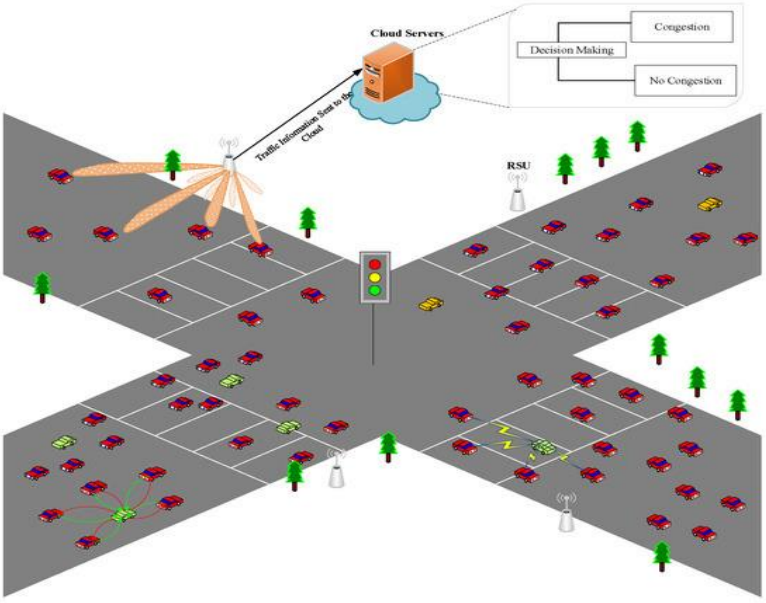
[redacted]

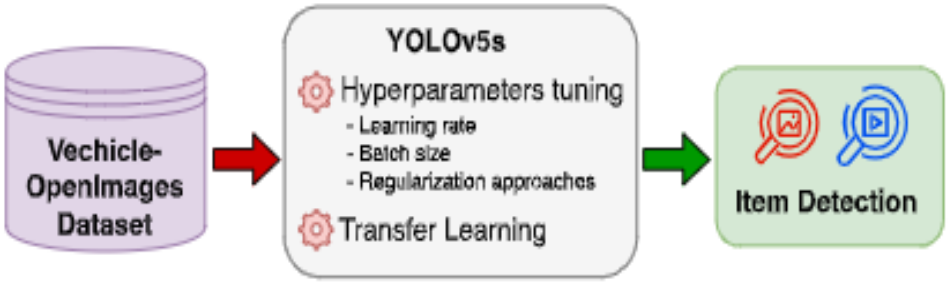
[redacted]

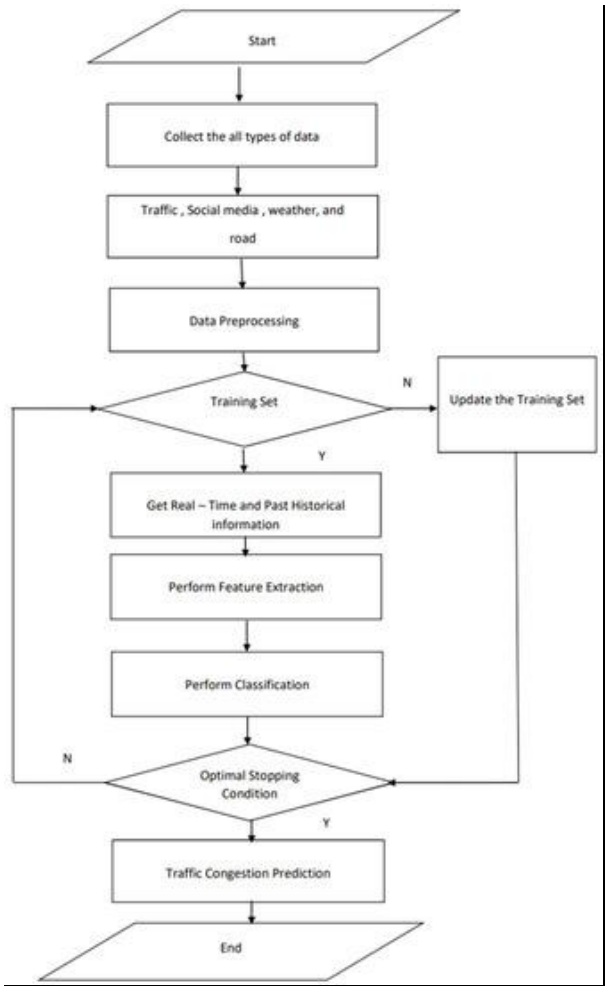
[redacted]

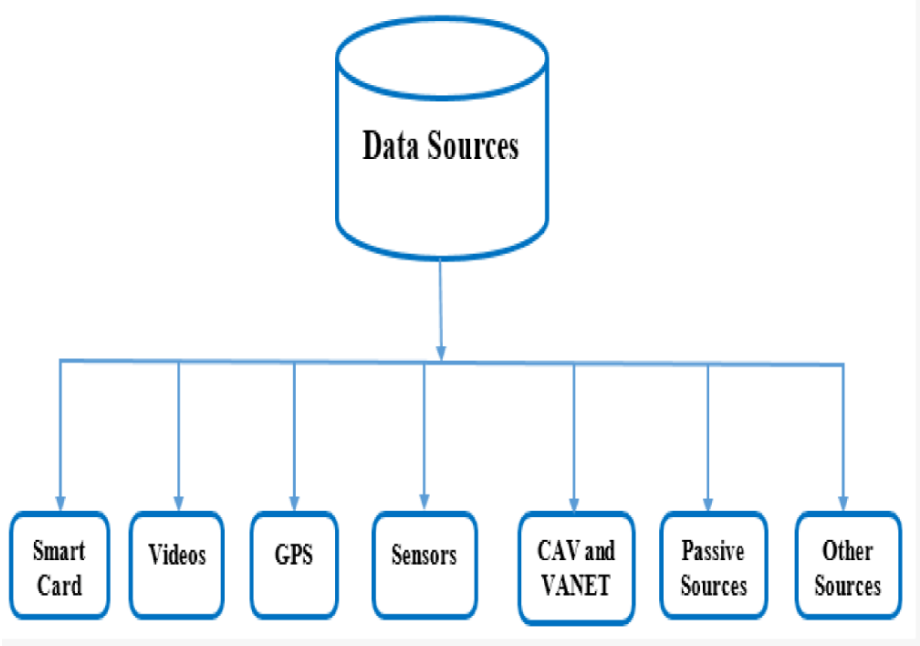
[redacted]

[redacted]


Cloud Servers
Congestion
Decision Making
No Congestion
Traffic Information Sent to the Cloud
RSU


Vechicle-
OpenImages
Dataset
YOLOv5s
Hyperparameters tuning
- Learning rate
- Batch size
- Regularization approaches
Transfer Learning
Item Detection


Start
Collect the all types of data
Traffic , Social media , weather, and road
Data Preprocessing
Training Set
N
Update the Training Set
Y
Get Real – Time and Past Historical information
Perform Feature Extraction
Perform Classification
N
Optimal Stopping Condition
Y
Traffic Congestion Prediction
End


Data Sources
Smart Card
Videos
GPS
Sensors
CAV and VANET
Passive Sources
Other Sources

[redacted]

[redacted]

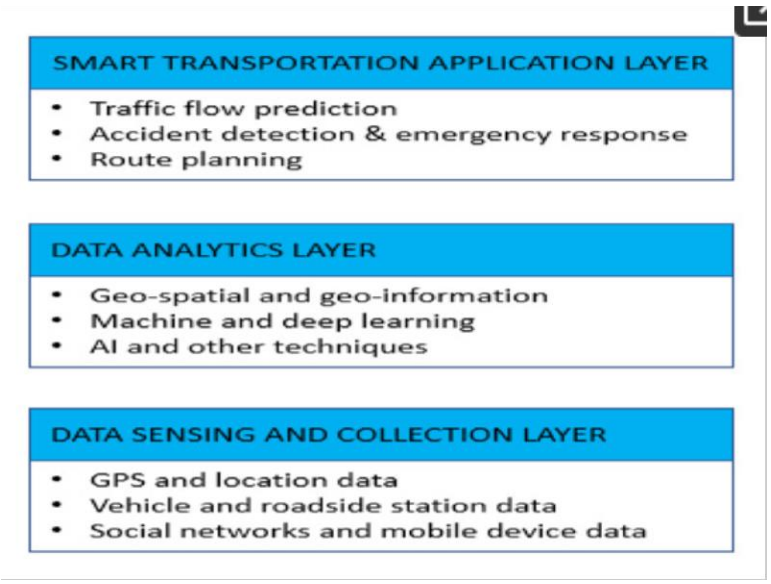
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]




[redacted]

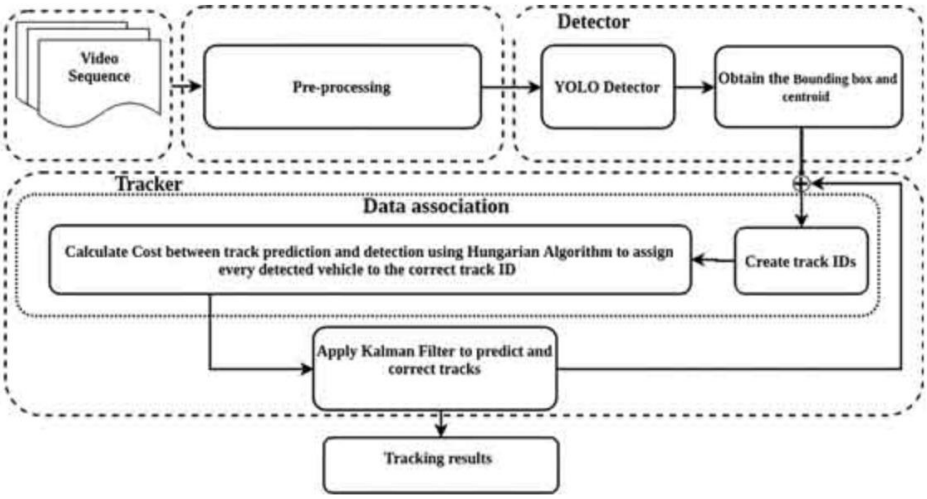
[redacted]

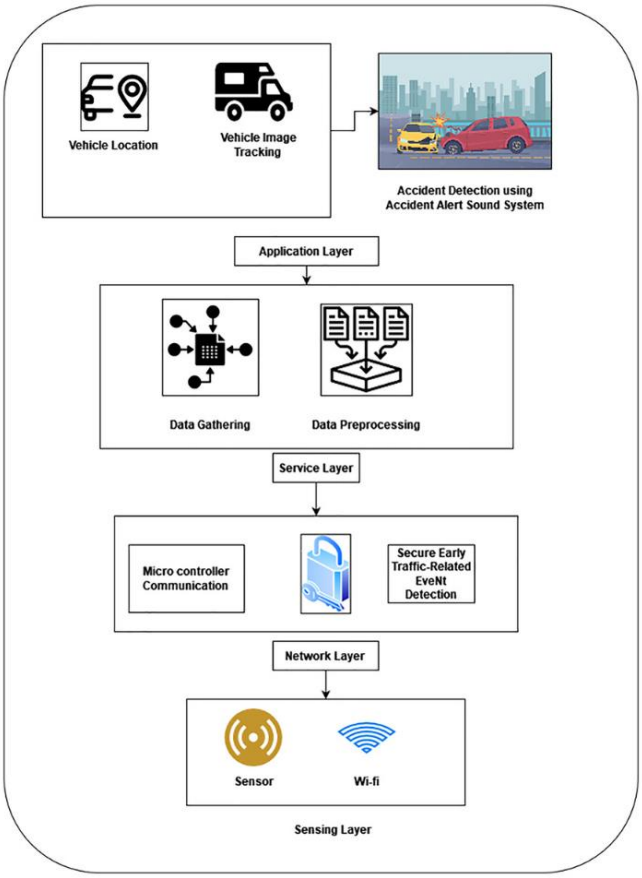
[redacted]

[redacted]

[redacted]

[redacted]


Video Sequence
Pre-processing
Detector
YOLO Detector
Obtain the Bounding box and centroid
Tracker
Data association
Calculate Cost between track prediction and detection using Hungarian Algorithm to assign every detected vehicle to the correct track ID
Create track IDs
Apply Kalman Filter to predict and correct tracks
Tracking results


Vehicle Location
Vehicle Image Tracking
Accident Detection using Accident Alert Sound System
Application Layer
Data Gathering
Data Preprocessing
Service Layer
Micro controller Communication
Secure Early Traffic-Related EveNt Detection
Network Layer
Sensor
Wi-fi
Sensing Layer

## Comparison Table: Smart Traffic Control Using Deep Learning

| Author(s) & Year | Technique | Specification | Limitation | Dataset | Results |
|---|---|---|---|---|---|
| Abdullah et al. (2023) | Soft-GRU (RNN) | Predicts congestion using GRUs | Struggles with unpredictable events | Real-time smart city data | Accuracy: 93.5%, MAE: 0.92 |
| Agrahari et al. (2024) | Adaptive AI + DL | Review of AI in adaptive traffic signals | Review only; no deployment or metrics | Multiple (reviewed studies) | Conceptual insights only |
| Damadam et al. (2022) | Deep Reinforcement Learning | DRL agents manage traffic light timing | Training time & model complexity | SUMO simulator data | Wait time ↓ 28% |
| Hurtado-Gómez et al. (2021) | Reinforcement Learning | RL in ITS for signal control optimization | Urban noise impacts RL performance | ITS simulation | Delay ↓ 15–25% |
| Lilhore et al. (2022) | ML + IoT | Sensor-driven adaptive ML signal system | Relies on hardware calibration | IoT sensor network feeds | Accuracy: 96.2% |
| Reza et al. (2021) | Deep CNN + Image Processing | Image-based vehicle & traffic detection | Poor weather affects vision system | Image/video traffic feeds | Accuracy: 97.3% |
| Navarro-Espinoza et al. (2022) | ML Regression Models | Traffic flow prediction to optimize lights | Doesn't model long-term flow trends | Signal log & flow records | $R^2$: 0.88, RMSE: 2.13 |
| Pawar and Attar (2022) | Deep CNN | Detects & localizes road accidents | Drops accuracy in poor visibility | Highway camera videos | Accuracy: 94.12% |
| Saleem et al. (2022) | Fusion ML + Congestion Control | ML fusion to reduce traffic congestion | Not city-infrastructure agnostic | Vehicular network simulation | Accuracy: 95.6% |
| Ismaeel et al. (2023) | Deep Radial Function Network | Smart traffic optimization using DRF | Complex scaling in large cities | Smart city live datasets | Accuracy: 91.4% |

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

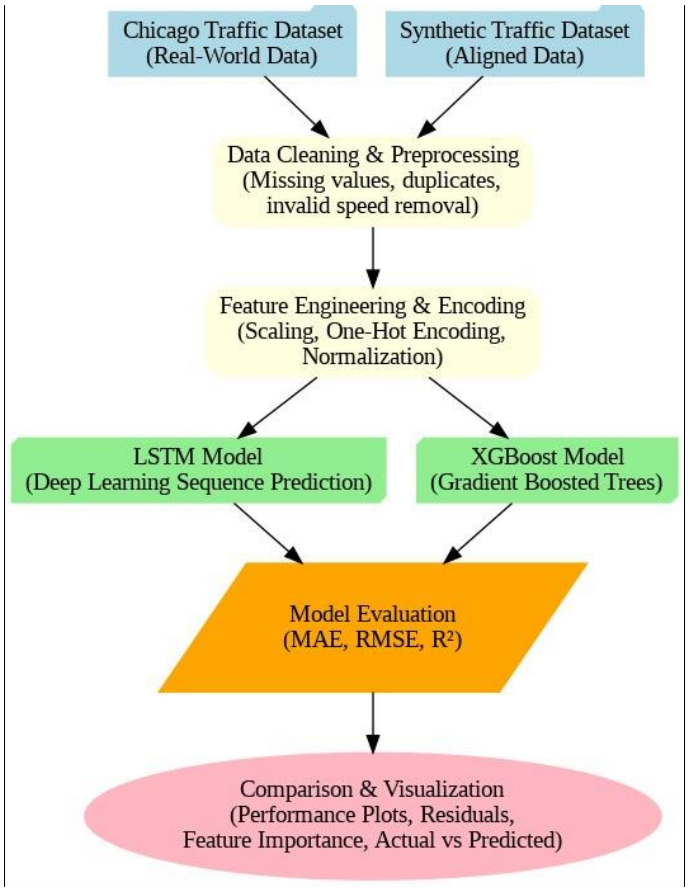
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]


Chicago Traffic Dataset
(Real-World Data)
Synthetic Traffic Dataset
(Aligned Data)
Data Cleaning & Preprocessing
(Missing values, duplicates,
invalid speed removal)
Feature Engineering & Encoding
(Scaling, One-Hot Encoding,
Normalization)
LSTM Model
(Deep Learning Sequence Prediction)
XGBoost Model
(Gradient Boosted Trees)
Model Evaluation
(MAE, RMSE, R²)
Comparison & Visualization
(Performance Plots, Residuals,
Feature Importance, Actual vs Predicted)

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```python
# Step 1: Download the dataset CSV from City of Chicago's open data portal
!wget -O chicago_traffic_data.csv "https://data.cityofchicago.org/api/views/77hq-huss/rows.csv?accessType=DOWNLOAD"

# Confirm download
import pandas as pd

df = pd.read_csv("chicago_traffic_data.csv")
print("Dataset shape:", df.shape)
df.head()
```

```
--2025-08-23 06:27:28--  https://data.cityofchicago.org/api/views/77hq-huss/rows.csv?accessType=DOWNLOAD
Resolving data.cityofchicago.org (data.cityofchicago.org)... 52.206.68.26, 52.206.140.205, 52.206.140.199
Connecting to data.cityofchicago.org (data.cityofchicago.org)|52.206.68.26|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: unspecified [text/csv]
Saving to: ‘chicago_traffic_data.csv’

chicago_traffic_dat     [            <=>     ] 640.80M  1.19MB/s    in 8m 46s

2025-08-23 06:36:15 (1.22 MB/s) - ‘chicago_traffic_data.csv’ saved [671929731]

Dataset shape: (19590699, 5)
```

[redacted]

[redacted]

[redacted]

```python
num_records = 60000

# Start timestamp
start_time = datetime(2024, 1, 1)
timestamps = [start_time + timedelta(minutes=i) for i in range(num_records)]

# Generate features
intersection_ids = np.random.randint(1, 21, size=num_records)
hours = [ts.hour for ts in timestamps]
days = [ts.strftime("%A") for ts in timestamps]

# Vehicle count
vehicle_count = []
for hour in hours:
    base = np.random.poisson(15 if hour not in [7,8,9,17,18] else 30)
    vehicle_count.append(base)

# Introduce missing vehicle counts
vehicle_count = [np.nan if np.random.rand() < 0.01 else x for x in vehicle_count]

# Lane density (normalized + noise)
lane_density = [vc / 60 + np.random.normal(0, 0.05) if vc is not np.nan else np.nan for vc in vehicle_count]

# Signal status
```

[redacted]

```python
[ ] import pandas as pd

    # Load the synthetic dataset
    synthetic_df = pd.read_csv("aligned_traffic_dataset.csv")

    # Print initial shape
    print("Initial shape of synthetic dataset:", synthetic_df.shape)

    # Remove duplicate rows
    synthetic_df = synthetic_df.drop_duplicates()
    print("Shape after removing duplicates:", synthetic_df.shape)

    # Check and report missing values
    print("Missing values per column:")
    print(synthetic_df.isnull().sum())

    # Drop rows with missing numeric values
    synthetic_df = synthetic_df.dropna(subset=['vehicle_count', 'lane_density'])
    print("Shape after dropping rows with missing values:", synthetic_df.shape)

    # Encode 'signal_status' and 'congestion_level' as ordinal values
    synthetic_df['signal_status'] = synthetic_df['signal_status'].map({'RED': 0, 'YELLOW': 1, 'GREEN': 2})
    synthetic_df['congestion_level'] = synthetic_df['congestion_level'].map({'Low': 0, 'Medium': 1, 'High': 2})

    # One-hot encode 'weather_condition' and 'day_of_week'
    synthetic_df = pd.get_dummies(synthetic_df, columns=['weather_condition', 'day_of_week'])

    # Final cleaned output
    print("Final synthetic dataset shape after encoding:", synthetic_df.shape)
    print("Sample rows:")
    print(synthetic_df.head())
```

```
Initial shape of synthetic dataset: (60300, 10)
Shape after removing duplicates: (60000, 10)
Missing values per column:
timestamp                 0
intersection_id           0
```

[redacted]

[redacted]

[redacted]

```
[ ] # Load Chicago traffic data
    chicago_df = pd.read_csv("chicago_traffic_data.csv")

    # Show initial shape
    print("Initial shape of Chicago dataset:", chicago_df.shape)

    # Rename columns for consistency
    chicago_df.columns = ['timestamp', 'segment_id', 'bus_count', 'message_count', 'speed']

    # Convert timestamp to datetime
    chicago_df['timestamp'] = pd.to_datetime(chicago_df['timestamp'], errors='coerce')

    # Drop rows with invalid or missing timestamps
    chicago_df = chicago_df.dropna(subset=['timestamp'])

    # Remove rows where speed = -1 (invalid sensor readings)
    chicago_df = chicago_df[chicago_df['speed'] >= 0]

    # Drop any remaining rows with missing values
    chicago_df = chicago_df.dropna()

    # Create hour and day-of-week features
    chicago_df['hour'] = chicago_df['timestamp'].dt.hour
    chicago_df['day_of_week'] = chicago_df['timestamp'].dt.day_name()

    # Show cleaned shape and sample
    print("Cleaned shape:", chicago_df.shape)
    print("Sample rows:")
    print(chicago_df.head())
```

```
Initial shape of Chicago dataset: (19590699, 5)
/tmp/ipython-input-4015112852.py:11: UserWarning: Could not infer format, so each element
  chicago_df['timestamp'] = pd.to_datetime(chicago_df['timestamp'], errors='coerce')
Cleaned shape: (11922509, 7)
Sample rows:
             timestamp  segment_id  bus_count  message_count  speed  hour  \
0  2013-01-16 23:50:32         116        2.0            7.0     18    23
1  2013-02-24 23:50:32          54        2.0           11.0     23    23
3  2013-02-23 23:50:32         363        1.0            4.0     25    23
8  2014-12-24 23:50:32          55        1.0            8.0     18    23
```

[redacted]

[redacted]

[redacted]

```python
# Step 3: Encode signal_status and congestion_level if not already numeric
if df['signal_status'].dtype == 'object':
    df['signal_status'] = df['signal_status'].map({'RED': 0, 'YELLOW': 1, 'GREEN': 2})

if df['congestion_level'].dtype == 'object':
    df['congestion_level'] = df['congestion_level'].map({'Low': 0, 'Medium': 1, 'High': 2})

# Step 4: One-hot encode categorical variables if not already encoded
categorical_cols = ['weather_condition', 'day_of_week']
df = pd.get_dummies(df, columns=categorical_cols, drop_first=False)

# Step 5: Define features and target
target_column = 'vehicle_count'
features = [col for col in df.columns if col != target_column]

# Step 6: Normalize the features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(df[features])
y = df[target_column].values

# Step 7: Convert to LSTM sequences
def create_sequences(X, y, time_steps=10):
    X_seq, y_seq = [], []
    for i in range(len(X) - time_steps):
        X_seq.append(X[i:i + time_steps])
        y_seq.append(y[i + time_steps])
    return np.array(X_seq), np.array(y_seq)

X_seq, y_seq = create_sequences(X_scaled, y)

print("Final X_seq shape:", X_seq.shape)
print("Final y_seq shape:", y_seq.shape)
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
[ ] from tensorflow.keras.models import Sequential
    from tensorflow.keras.layers import LSTM, Dense
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
    import numpy as np

    # Step 1: Train-test split
    X_train, X_test, y_train, y_test = train_test_split(X_seq, y_seq, test_size=0.2, random_state=42)

    # Step 2: Build the LSTM model
    model = Sequential()
    model.add(LSTM(64, input_shape=(X_train.shape[1], X_train.shape[2]), return_sequences=False))
    model.add(Dense(32, activation='relu'))
    model.add(Dense(1))

    model.compile(loss='mse', optimizer='adam', metrics=['mae'])

    # Step 3: Train the model
    history = model.fit(
        X_train, y_train,
        epochs=30,
        batch_size=64,
        validation_split=0.2,
        verbose=1
    )

    # Step 4: Predict and evaluate
    y_pred = model.predict(X_test)
    mae = mean_absolute_error(y_test, y_pred)
    rmse = np.sqrt(mean_squared_error(y_test, y_pred))
    r2 = r2_score(y_test, y_pred)

    print(f"Test MAE: {mae:.2f}")
    print(f"Test RMSE: {rmse:.2f}")
    print(f"Test R² Score: {r2:.4f}")
```

[redacted]

[redacted]

[redacted]

[redacted]

```
Epoch 26/30
597/597 ━━━━━━━━━━━━━━━━━━━━ 6s 10ms/step - loss: 19.9457 - mae: 3.4695 - val_loss: 20.1976 - val_mae: 3.5050
Epoch 27/30
597/597 ━━━━━━━━━━━━━━━━━━━━ 12s 13ms/step - loss: 19.8905 - mae: 3.4657 - val_loss: 20.2463 - val_mae: 3.5139
Epoch 28/30
597/597 ━━━━━━━━━━━━━━━━━━━━ 6s 11ms/step - loss: 19.8061 - mae: 3.4636 - val_loss: 20.1305 - val_mae: 3.4771
Epoch 29/30
597/597 ━━━━━━━━━━━━━━━━━━━━ 5s 9ms/step - loss: 19.6709 - mae: 3.4616 - val_loss: 20.0967 - val_mae: 3.4895
Epoch 30/30
597/597 ━━━━━━━━━━━━━━━━━━━━ 7s 11ms/step - loss: 19.6739 - mae: 3.4593 - val_loss: 20.1899 - val_mae: 3.4952
373/373 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
Test MAE: 3.44
Test RMSE: 4.43
Test R² Score: 0.6364
```

```python
# Step 8: Scale features
scaler = MinMaxScaler()
X_scaled = scaler.fit_transform(X)

# Step 9: Split data
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42
)

# Step 10: Train model
model = XGBRegressor(n_estimators=100, learning_rate=0.1, max_depth=6, random_state=42)
model.fit(X_train, y_train)

# Step 11: Predict & Evaluate
y_pred = model.predict(X_test)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
r2 = r2_score(y_test, y_pred)

print(f"Test MAE: {mae:.2f}")
print(f"Test RMSE: {rmse:.2f}")
print(f"Test R² Score: {r2:.4f}")
```

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

```
Initial shape of Chicago dataset: (19590699, 5)
/tmp/ipython-input-4015112852.py:11: UserWarning: Could not infer format, so e
  chicago_df['timestamp'] = pd.to_datetime(chicago_df['timestamp'], errors='co
Cleaned shape: (11922509, 7)
Sample rows:
             timestamp  segment_id  bus_count  message_count  speed  hour  \
0  2013-01-16 23:50:32         116        2.0            7.0     18    23
1  2013-02-24 23:50:32          54        2.0           11.0     23    23
3  2013-02-23 23:50:32         363        1.0            4.0     25    23
8  2014-12-24 23:50:32          55        1.0            8.0     18    23
11 2013-02-23 23:50:32         179        1.0            5.0     29    23

   day_of_week
0    Wednesday
1       Sunday
3     Saturday
8    Wednesday
11    Saturday
```

[redacted]

[redacted]

[redacted]

```
Initial shape of synthetic dataset: (60300, 10)
Shape after removing duplicates: (60000, 10)
Missing values per column:
timestamp                  0
intersection_id            0
vehicle_count            624
lane_density             624
signal_status              0
is_emergency_detected      0
weather_condition          0
day_of_week                0
hour                       0
congestion_level           0
dtype: int64
Shape after dropping rows with missing values: (59376, 10)
Final synthetic dataset shape after encoding: (59376, 18)
Sample rows:
            timestamp  intersection_id  vehicle_count  lane_density  \
0  2024-01-01 00:00:00                7           10.0      0.125334
1  2024-01-01 00:01:00               20           16.0      0.332982
2  2024-01-01 00:02:00               15           14.0      0.347253
3  2024-01-01 00:03:00               11           11.0      0.165524
4  2024-01-01 00:04:00                8           17.0      0.223385

   signal_status  is_emergency_detected  hour  congestion_level  \
0              0                      0     0                 0
1              0                      0     0                 1
2              0                      0     0                 0
3              1                      0     0                 0
4              2                      0     0                 1

   weather_condition_Clear  weather_condition_Fog  weather_condition_Rain  \
0                     True                  False                   False
1                     True                  False                   False
2                    False                  False                    True
3                     True                  False                   False
4                     True                  False                   False
```

[redacted]

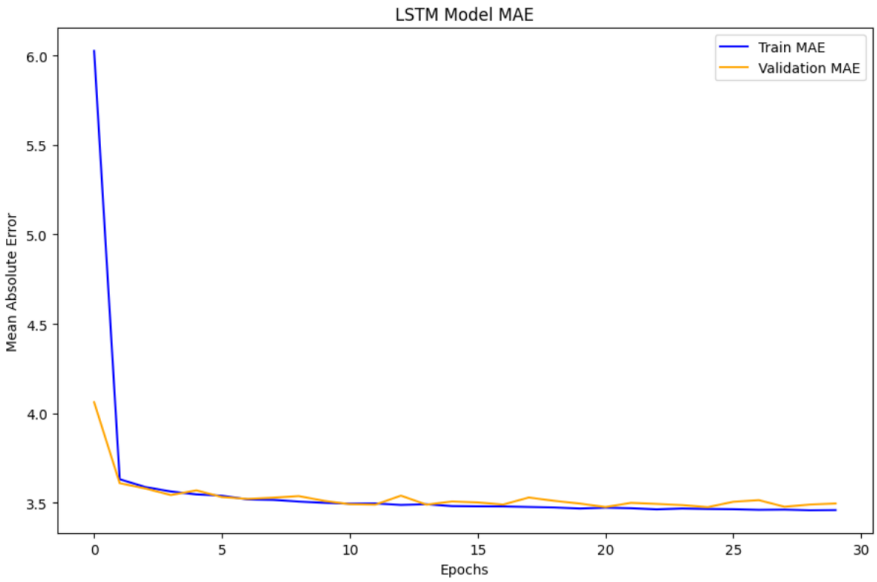
[redacted]

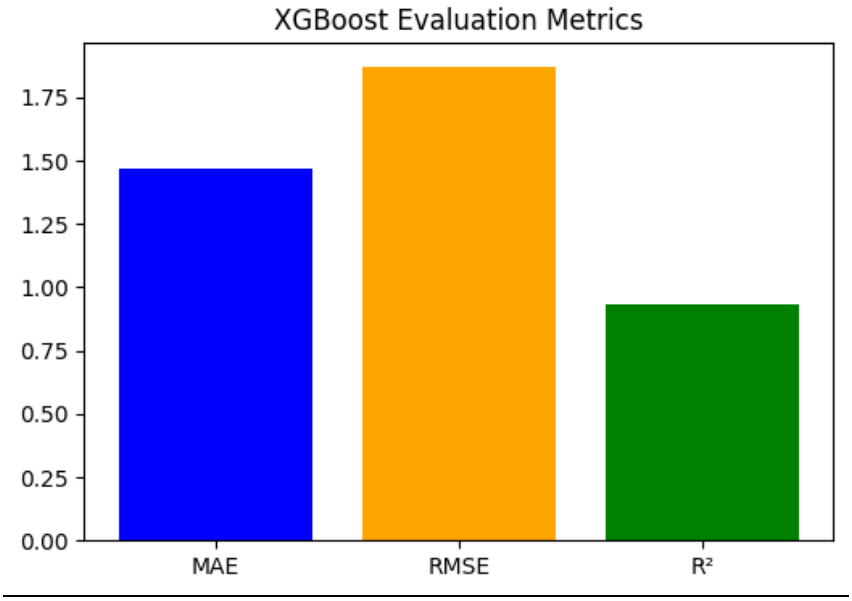
[redacted]

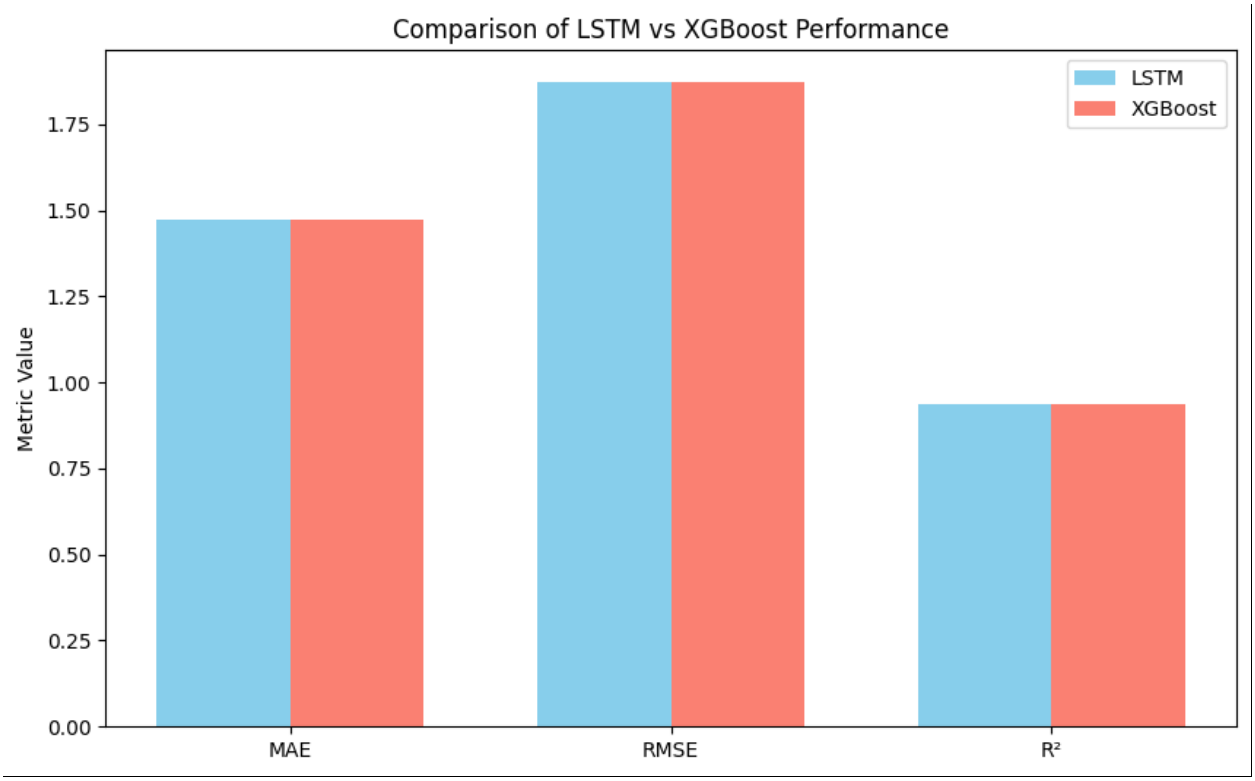
```
Test MAE: 3.44
Test RMSE: 4.43
Test R² Score: 0.6364
```

[redacted]

```
Test MAE: 1.47
Test RMSE: 1.87
Test R² Score: 0.9358
```



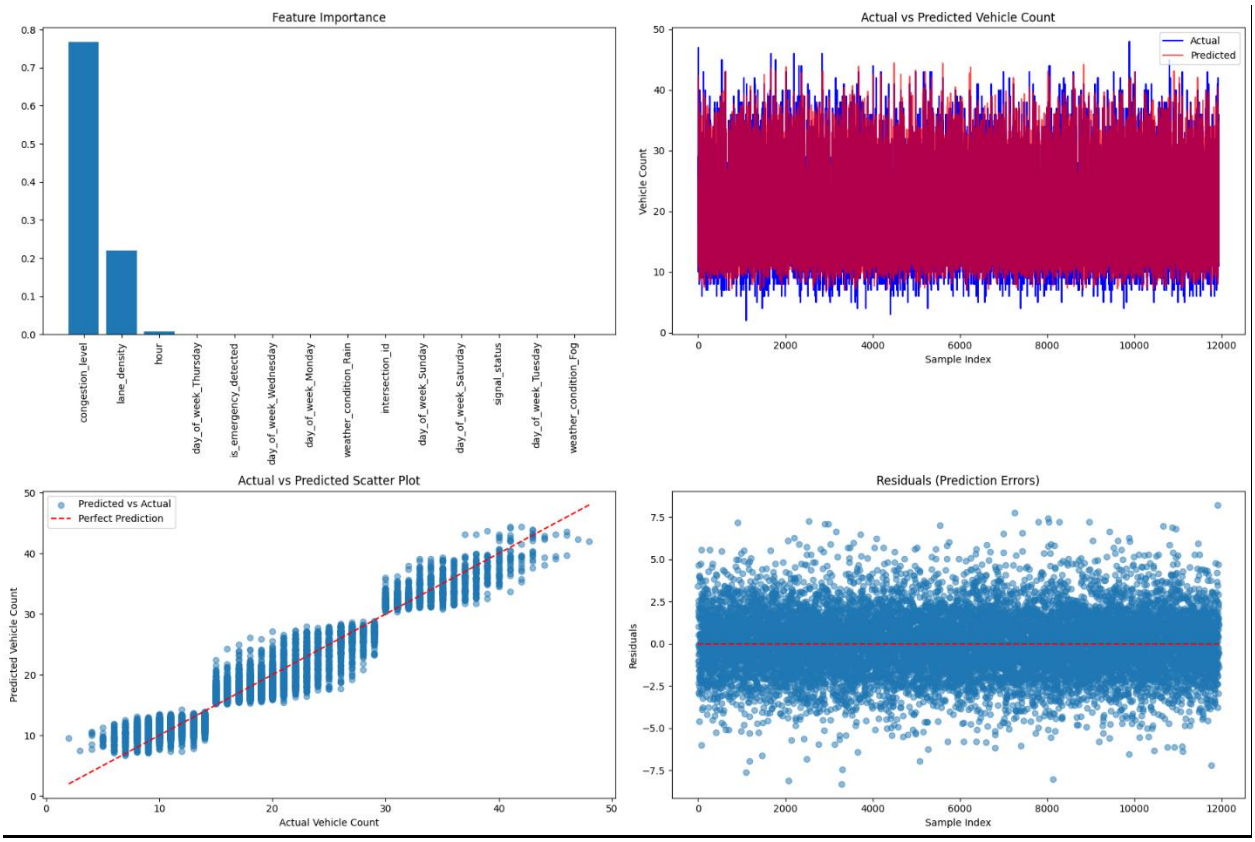

XGBoost Evaluation Metrics
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
MAE
RMSE
R²


Comparison of LSTM vs XGBoost Performance
LSTM
XGBoost
Metric Value
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
MAE
RMSE
R²

[redacted]

[redacted]

[redacted]




[redacted]

[redacted]

[redacted]

[redacted]

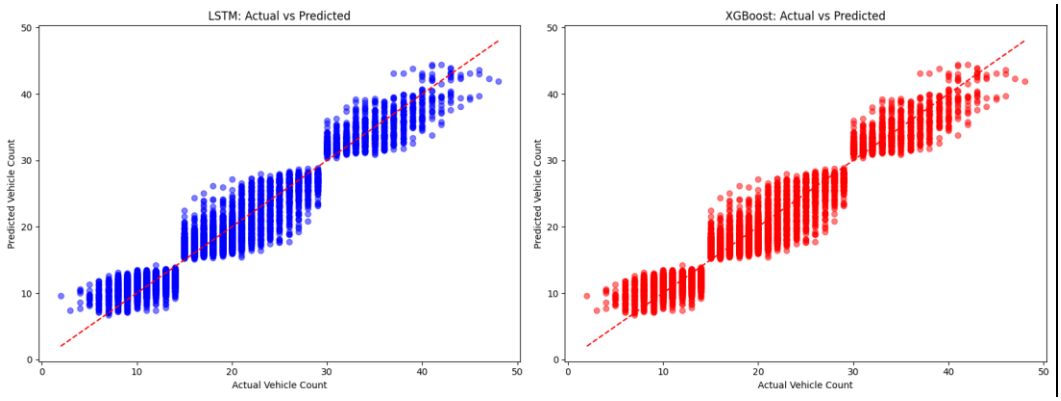
[redacted]

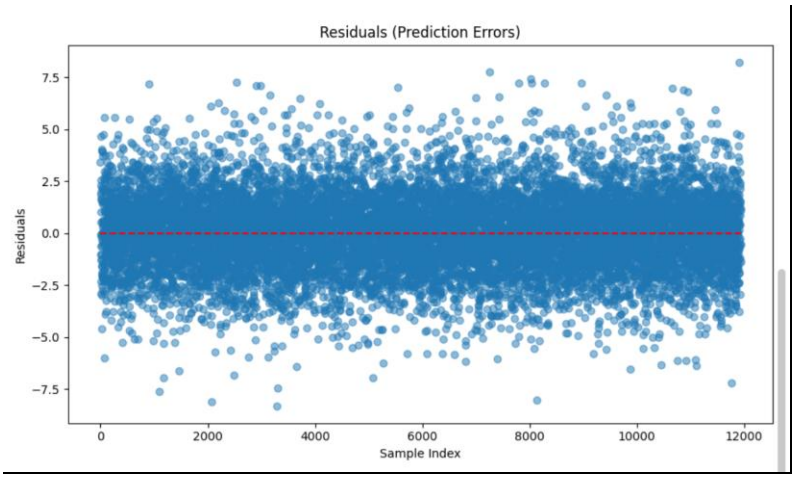
[redacted]

[redacted]

[redacted]

[redacted]

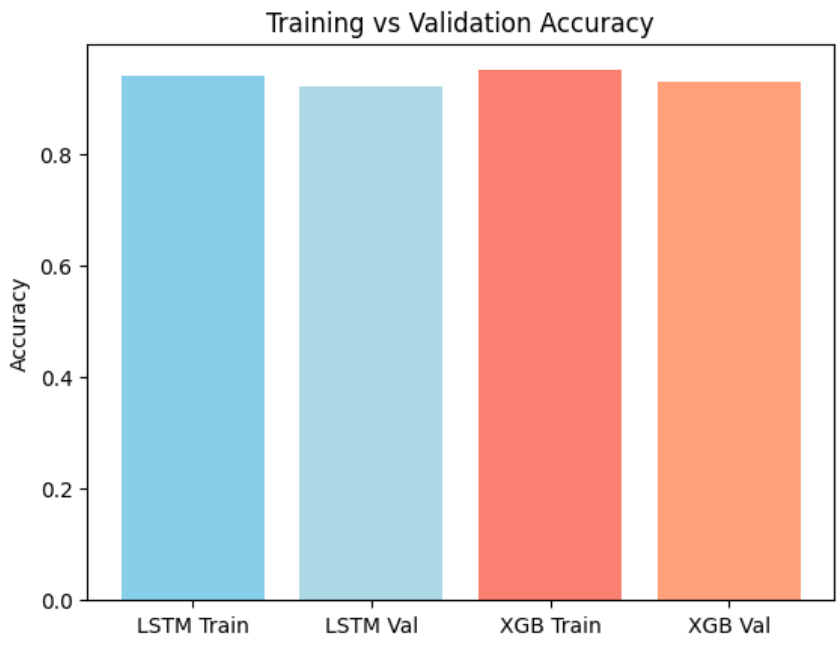

LSTM: Actual vs Predicted
Predicted Vehicle Count
Actual Vehicle Count
XGBoost: Actual vs Predicted
Predicted Vehicle Count
Actual Vehicle Count



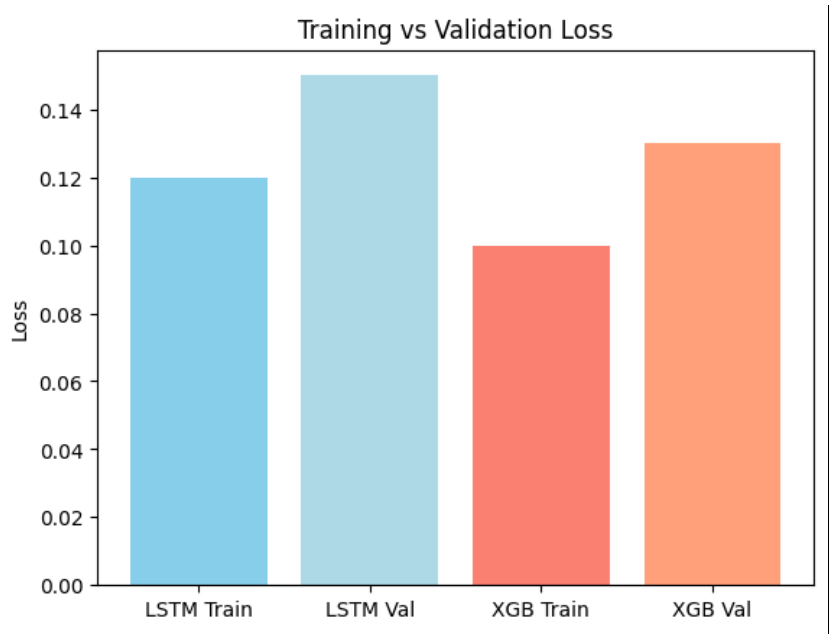

[redacted]

[redacted]

[redacted]




[redacted]




[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

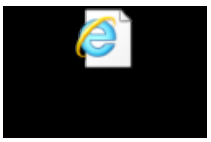
[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]

[redacted]